

Name: _____

Date: _____

Tidy up the Media Club archive from the terminal. The archive is a mess. Your task is to sort it out with commands only. Do not use the mouse.

You get one zip file. Task 1 gives you the exact commands that unpack it. After that you type every move, copy, rename, search and count. You do not need the mouse at any point in this assignment.

The commands and the messages here are the ones your Ubuntu laptop prints. Ubuntu runs the *bash* shell. Another computer can word the same message a little differently.

Do the parts in order. Each part uses the work of the part before it. Part 2 copies files into folders that you make in Part 1. Part 3 fills the gallery folder that Part 2 leaves empty. Part 5 counts what Part 3 finds.

Write your answers on this sheet as you work. The ruled lines and the tables are part of the task.

What is inside `cs17-archive.zip`. Task 1 unpacks the zip. You get one folder with the name `cs17-archive`. Every file and folder below is inside it. Keep this map beside you. Most of the work on this sheet is to reach the correct place in this tree.

```
cs17-archive/
|-- README.txt
|-- notes.txt
|-- club-members-attendance-register.txt
|-- inbox/
|   |-- Unsorted Camera Dump/
|   |   |-- IMG_0001.jpg
|   |   |-- IMG_0002.jpg
|   |   |-- IMG_0003.jpg
|   |   |-- IMG_0004.jpg
|   |   |-- IMG_0005.jpg
|   |   |-- IMG_0006.jpg
|   |   |-- scoreboard-screenshot.png
|   |   |-- camera-settings-backup.txt
|   |-- competitions/
|   |   |-- photography-competition-entries-2026/
|   |   |   |-- entry-aisha-sunrise.jpg
|   |   |   |-- entry-rohan-bridge.jpg
|   |   |   |-- entry-meera-street-cat.jpg
|   |   |-- entries-received.txt
|-- documents/
|   |-- reports/
|   |   |-- term-1-science-fair-report.txt
|   |   |-- term-2-science-fair-report.txt
|   |-- minutes/
|   |   |-- meeting-minutes-2026-01-14.txt
|   |   |-- meeting-minutes-2026-02-11.txt
|   |-- drafts/
|   |   |-- old-notes.txt
|   |   |-- scrap.txt
|-- logs/
|   |-- experiment-log-week-01.txt
|   |-- experiment-log-week-02.txt
|   |-- experiment-log-week-03.txt
|   |-- experiment-log-week-04.txt
|-- gallery/
|-- backup/
```

Three habits. Use them from the first command to the last.

- **Look before you act.** Run `pwd` and `ls` before every command that copies, moves or deletes. There is no Undo. There is no Trash.
- **Press Tab. Do not type the whole name.** Many names in this archive are long. Type three letters, then press Tab.
- **Read the error.** An error message tells you what went wrong. Read the whole line before you type the command again.

Part 1. Moving around the archive

Lesson 1

Ask two questions before every command. Where am I? What is here? Each question has one command. You will type these two commands more often than any other command in this assignment.

Commands this part uses

<code>pwd</code>	print the absolute path of the folder that you are in
<code>ls</code>	list the contents of this folder
<code>ls -l</code>	list with the size, the date and the permissions
<code>ls -a</code>	list all entries, including the hidden ones
<code>cd folder</code>	go into a folder
<code>cd ..</code>	go up one folder, to the parent
<code>cd ~</code>	go to your home folder
<code>mkdir name</code>	make a new folder
<code>unzip file.zip</code>	unpack a zip file into the folder that you are in

1. Open the terminal. The archive is in your `Downloads` folder. Run these five commands, in this order. They make a folder on your Desktop, move the zip into it, and unpack it. You meet `mkdir`, `mv` and `cd` again later in this assignment. `unzip` you need only here.

Hint: Add `-q` to `unzip` to unpack the archive quietly, with no list of files.

```
mkdir ~/Desktop/terminal-assignment
mv ~/Downloads/cs17-archive.zip ~/Desktop/terminal-assignment
cd ~/Desktop/terminal-assignment
unzip cs17-archive.zip
ls
```

2. Go into the `cs17-archive` folder. Print the folder that you are in. Copy the address exactly as the shell prints it.

Hint: This address is the absolute path of the archive. Task 10 needs it again.

3. List the contents of the archive folder. How many items are there?

4. List the folder again with the long details. Name two things that the long listing shows and the plain listing does not.

-
-
5. List all entries, including the hidden ones. Write down one entry that the earlier listings did not show.

 6. Start in the archive folder. Go into the reports folder with one command and a relative path. Print the folder to check the result.

 7. Go back up from reports to the archive folder. Use one command.

 8. Start in the archive folder. Go into the deepest folder inside `inbox` with one relative path. Press Tab for the long names.

 9. Stay in that deep folder. Go to the `logs` folder with a relative path. Do not use `~`. Do not use an absolute path.

 10. Go to your home folder. Then go back to the archive with one command. Use the absolute path from task 2.

 11. Go to the archive folder. Make a folder with the name `sorted`. Then make two folders inside it, `images` and `notes`, with one command.

The path drill. Each row asks the same question two times. Write the relative path first. A relative path gives directions from the folder that you are in. Then write the absolute path. An absolute path is the full address and it starts at your home folder `~`. The archive is at `~/Desktop/terminal-assignment/cs17-archive`.

You are in	You want to reach	Relative path	Absolute path
cs17-archive	logs		
cs17-archive/logs	cs17-archive/ documents		
documents/reports	inbox		

You are in	You want to reach	Relative path	Absolute path
inbox/Unsorted Camera Dump	logs		
inbox/competitions/ photography- competition-entries- 2026	gallery		
documents/minutes	documents/reports		
gallery	cs17-archive		
anywhere at all	your home folder		

12. Read your finished table. Write one sentence about when to use a relative path. Write one sentence about when to use an absolute path.

Remember

`..` is not a normal folder. It is a shortcut and it always means the folder above this one. `.` means this folder. `ls -a` shows both of them.

Part 2. Files without a GUI: read, make, copy, move, delete

Lesson 2

Every action that you do with a drag or a right-click has a command. Start each task in the archive folder, unless the task says something else. Run `pwd` to check where you are.

Commands this part uses

<code>cat file</code>	show the contents of a file on the screen
<code>touch file</code>	make an empty file
<code>echo "text"</code>	show some text on the screen
<code>echo "text" > file</code>	write that text into a file and replace the old contents
<code>cp from to</code>	copy a file
<code>mv from to</code>	move a file, or rename it
<code>rm file</code>	delete a file. There is no Trash and no Undo
<code>rmdir folder</code>	delete a folder, but only if the folder is empty

1. Show `README.txt` on the screen. Do not open an application. What is the last line?

2. Show the Term 1 science fair report. Do not change folder first. Use one command and one relative path. How many rupees came out of the budget?

3. Make an empty file in the archive folder with the name `tidy-up-log.txt`. Then show that the file is there.
4. Write one line into that file. The line has your name and the word *started*. Then show the file to check it.
5. Copy the Term 2 report into `sorted/notes`. The original file must stay in its place.
6. Move `old-notes.txt` from `documents/drafts` into `sorted/notes`.
7. Rename that file to `last-year-notes.txt`. Keep it in `sorted/notes`.
8. Task 5 uses `cp` and task 6 uses `mv`. Write the difference in one sentence. Then write which command makes a backup.

9. Delete `scrap.txt` from `documents/drafts`. Run the two check commands from the habits box first. Write all three commands on the lines below.

10. `documents/drafts` is now empty. Delete the empty folder.
11. Try to delete `documents` with the same command. Copy the message from the shell. Explain why the command failed.

12. List `sorted/notes` with the long details. Which of the two files is bigger? How do you know?

Remember

`rm` does not put a file in the Trash. You cannot restore the file and you cannot undo the command. For this reason, run `pwd` and `ls` first every time. For the same reason, this assignment does not use `rm -rf`.

A good terminal user does not type everything. Four habits make the terminal faster than the mouse. The shell finishes your names. The shell keeps the commands that you typed before. One command works on a whole group of files. One command searches inside a file without opening it.

Commands this part uses

<code>Tab</code>	finish the name that I started to type
<code>Up / Down</code>	move back and forward through earlier commands
<code>history</code>	list the commands that you ran
<code>clear</code>	clear the screen. <code>Ctrl+L</code> does the same
<code>"name with spaces"</code>	quote a name so the shell reads it as one name
<code>*</code>	match any text here. <code>*.jpg</code> means every name that ends in <code>.jpg</code>
<code>grep word file</code>	show every line of the file that contains word
<code>wc file</code>	count the lines, words and bytes
<code>wc -l file</code>	count the lines only

1. Start in the archive folder. Type `cat club` and stop there. Press Tab. Write the full name that the shell completes. Then press Enter and look at the file.

2. Go into `documents/minutes`. Type `cat meeting-minutes-2026-0` and press Tab. Nothing happens. Press Tab a second time. Explain why the first Tab did not complete the name.

3. Go back to `inbox`. Enter the `Unsorted Camera Dump` folder in three ways. Write the result of each one. (a) Type the name with no quotes and no Tab, then copy the error. (b) Put the whole name in double quotes. (c) Type `Uns` and press Tab, then copy what Tab typed for you.

4. Show the commands that you ran so far. How many commands did the shell record? Then press the Up arrow to find the `pwd` from Part 1. Run it again without typing it.

5. Clear the screen. Your commands are still there. Press the Up arrow to check.

Predict before you press Enter. Start in the archive folder. Write what you expect each line to print. Then run the line and correct your answer.

Command	Write your prediction, then run the line
<code>ls *.txt</code>	
<code>ls *.jpg</code>	
<code>ls inbox/*</code>	
<code>ls logs/*week-0*</code>	

6. List every `.jpg` file inside the camera dump. How many are there?

7. Copy every `.jpg` file from the camera dump into `gallery`. Use one command. Then count the files in `gallery`.

8. Copy all four experiment logs into `backup`. Use one command.

9. Search the week 1 experiment log for the word `SUCCESS`. How many days passed?

10. Search all four logs with one command. Which week has the most days with `SUCCESS`?

11. Search the Term 2 report for `Scratch` with a capital S. Then search it for `scratch` with a small s. Are the lines the same? What does this show about `grep`?

12. Count the lines, words and bytes in the attendance register. Write the three numbers. Which number is the count of register entries?

13. Count the lines only. Then find how many lines contain `Aisha`. Count them by hand from the output of `grep`.

14. Count the lines in all four logs with one command. What is the total? Where does the shell show the total?

Remember

The shell expands `*`, not the command. The shell gives `cp` all six filenames before `cp` starts. For this reason, check what a `*` matches before you use it with a command that deletes.

Part 4. Help, errors and safe habits

Lesson 5

Everybody makes typing mistakes. A confident terminal user is not a user who never makes a mistake. A confident user reads the error and then recovers. In this part you break commands on purpose.

Commands this part uses

<code>command --help</code>	ask a command to list its own options
<code>man command</code>	open the full manual. Press <code>q</code> to quit
<code>pwd</code>	run this before a command that deletes. Where am I?
<code>ls</code>	run this before a command that deletes. Is that the correct file?

1. Run `ls --help`. The page is long. Do not read all of it. Find the line for one option that you already use, and write that line down.

2. Open the manual for `ls`. Find what the `-a` option does. Write the answer in your own words. Press `q` to quit.

Break it on purpose. Start in the archive folder. Run each line exactly as it is written. Copy the important part of the message from the shell. Then write in a few words why the command failed.

Run this	What the shell printed	Why it failed
<code>cd documets</code>		
<code>cd notes.txt</code>		
<code>cat notes</code>		
<code>cp missing.txt backup/</code>		
<code>rmdir logs</code>		
<code>cd inbox/Unsorted Camera Dump</code>		

3. Capital letters, part one. This part is the same on every computer. Run `ls -l`. Then run `ls -L`. The letter is the same and the case is different. Are the two listings the same?

4. Capital letters, part two. Names. Run `cat NOTES.txt`. Then run a command name in capitals, `LS`. Copy both messages. The file `notes.txt` is there and the command `ls` exists, so why does Ubuntu refuse both?

5. Safety drill. You are going to delete `gallery/IMG_0006.jpg`. The file is a copy, so this is safe. Write the two check commands that come before the delete. Then run all three commands.

6. A command failed and you do not know why. Write the five checks that you make, in order, before you ask for help.

Remember

You can find `rm -rf` on the internet. Do not use it. It deletes a whole folder tree and it asks no questions. In the wrong folder it deletes your work. This assignment needs `rm` and `rmdir` only.

Part 5. Combining commands: pipes and redirection

Lesson 4 (not taught yet)

This is next week's lesson. Try it now. Each command that you know does one small job. `cat` shows a file. `grep` selects lines. `wc` counts. The shell joins these jobs end to end. The output of one command becomes the input of the next command:

INPUT → COMMAND → OUTPUT

`|` passes the output to another command. `>` writes the output into a file and not to the screen. `>>` adds the output to the end of a file. Predict the result of each line before you press Enter.

Commands this part uses

`a | b` give the output of a to b as input
`a > file` write the output of a into file and replace the contents
`a >> file` add the output of a to the end of file and keep the rest

1. Show the week 1 experiment log. Pass the output to `grep` so that only the `SUCCESS` lines appear.
2. Add one more stage. The shell must print the number of successful days and not the lines.

3. Write one sentence. What does the `|` do in that command?

4. Save the listing of `gallery` into a file with the name `gallery-list.txt`. Put the file in the archive folder. Then show the file to check it.
5. Add one line to the end of that file with the name of the person who checked it. Do not delete the lines that are already there. Then show the file.
6. Run the same line again with a single `>` and not `>>`. Then show the file. What happened to the filenames? Build the file again in the correct way. The last section needs this file.

7. How many test days failed in all four logs? Answer with one command. Use `cat`, `grep` and `wc`.

8. How many lines of the attendance register contain `Aisha`? Do not print the lines. Save only the number into a file with the name `aisha-count.txt`. Then show that file.

One more prediction round. Write your answer first. Then run the line and compare it with your answer.

Command	Write your prediction, then run the line
<code>cat logs/*.txt wc -l</code>	
<code>grep present club-members-attendance-register.txt wc -l</code>	

Command	Write your prediction, then run the line
<code>grep absent club-members-attendance-register.txt wc -l</code>	
<code>cat documents/reports/*.txt grep budget wc -l</code>	

Remember

Read a pipeline from left to right, like a sentence: *show the logs, keep the failures, count them*. If a long pipeline gives a strange result, remove the last stage and run it again. This shows you what passes through the middle.

Hand in

Build your hand-in from the terminal. Do this section last. Start in the archive folder.

1. Make a folder with the name `submission` inside `cs17-archive`.
2. Move `gallery-list.txt`, `aisha-count.txt` and `tidy-up-log.txt` into it. Use one command for all three files.
3. Write your full name into `submission/name.txt`.
4. List `submission` with the long details. Check that four files are there.
5. Go up one folder, to `terminal-assignment`. Pack the whole `cs17-archive` folder into a new zip file. Put your own name in the filename. `zip` is the partner of `unzip`, and `-r` tells it to include every folder inside:

```
cd ~/Desktop/terminal-assignment
zip -r submission-your-name.zip cs17-archive
```

6. Upload that zip file. Hand in your answers with it.

Check this list before you hand in.

- `sorted/notes` holds `last-year-notes.txt` and a copy of the Term 2 report.
- `documents/drafts` is deleted.
- `gallery` holds the camera dump photographs, without the file that you deleted in Part 4.
- `backup` holds all four experiment logs.
- `submission` holds four files. One of them has your name.
- Every question and every table has an answer.

Command reference

<code>pwd</code>	show the folder that you are in
<code>ls</code>	list the contents of this folder
<code>ls -l</code>	list with the sizes and the dates
<code>ls -a</code>	list the hidden entries too
<code>cd</code>	change folder
<code>mkdir</code>	make a folder
<code>touch</code>	make an empty file
<code>cat</code>	show the contents of a file
<code>cp</code>	copy a file
<code>mv</code>	move a file, or rename it
<code>rm</code>	delete a file. This is permanent
<code>rmdir</code>	delete an empty folder
<code>echo</code>	show text on the screen
<code>clear</code>	clear the screen
<code>history</code>	show the commands that you ran
<code>grep</code>	show the lines of a file that contain a word
<code>wc</code>	count the lines, words and bytes
<code>wc -l</code>	count the lines only
<code>unzip</code>	unpack a zip file
<code>zip -r</code>	pack a folder and everything inside it into a zip file
<code>man</code>	open the manual of a command. Press q to quit
<code>--help</code>	ask a command to list its options
<code>></code>	write output into a file and replace the contents
<code>>></code>	add output to the end of a file
<code> </code>	send the output of one command into another command
<code>*</code>	match any text here
<code>..</code>	the folder above this one
<code>~</code>	your home folder
<code>"name with spaces"</code>	one name, not several arguments
<code>Tab</code>	finish the name that I started to type
<code>Up / Down</code>	move through the commands that you ran